객체지향 프로그래밍 기초

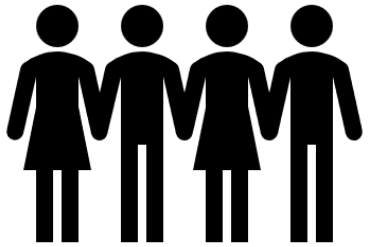
2강. C++ 시작하기

2강. 객체지향 프로그래밍 개요

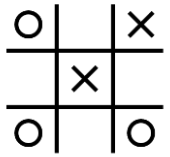
프로그램 언어의 개념

프로그래밍 언어

- 사람이 컴퓨터에게 일을 시키기 위해서는 컴퓨터와 사람의 대화를 위한 공통의 대화수단이 필요하다.
- 컴퓨터는 0과 1로 된 기계어만을 이해하고 사람은 사람은 기계어를 이해하기 힘들다.
- 사람은 이해하기 쉬운 프로그래밍 언어로 컴퓨터에게 전달하고, 컴퓨터는 '컴파일러 ' 를 통해 기계어로 통역하고 이해한다.



"게임프로그램을 만들고싶다"





```
#include "stdafx.h"
#include "stdlib.h"
#define SIZE 5

int _tmain(int argc, _TCHAR* argv[])
{
    char str[20] = "hello world!"; // 문자배열 선
    char *ptr = "good bye";      // 문자열 선언

    printf("대입이전\n %s\n %s\n\n", str, ptr);
    ptr = str;  // ptr에 str 대입
    printf("대입이후\n %s\n %s\n", str, ptr);

    system("pause");
```

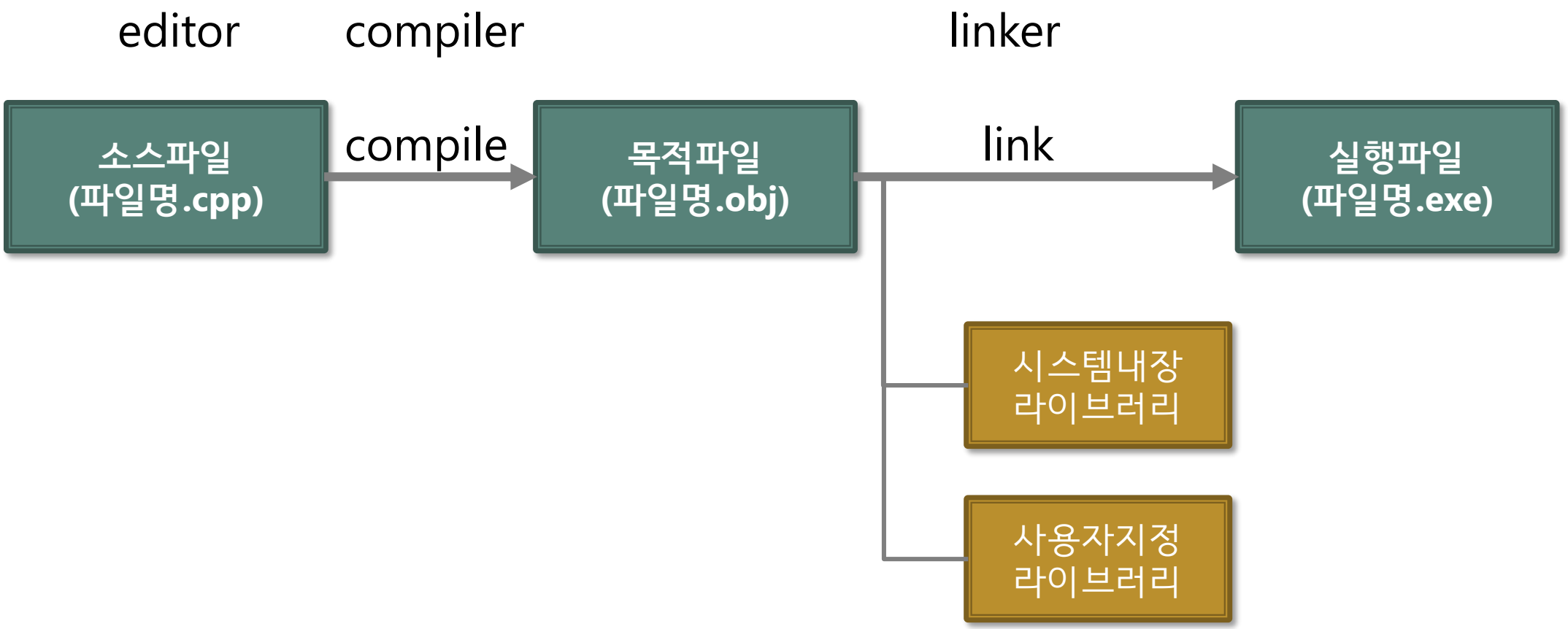
사람이 이해하기 쉬운 프로그래밍언어



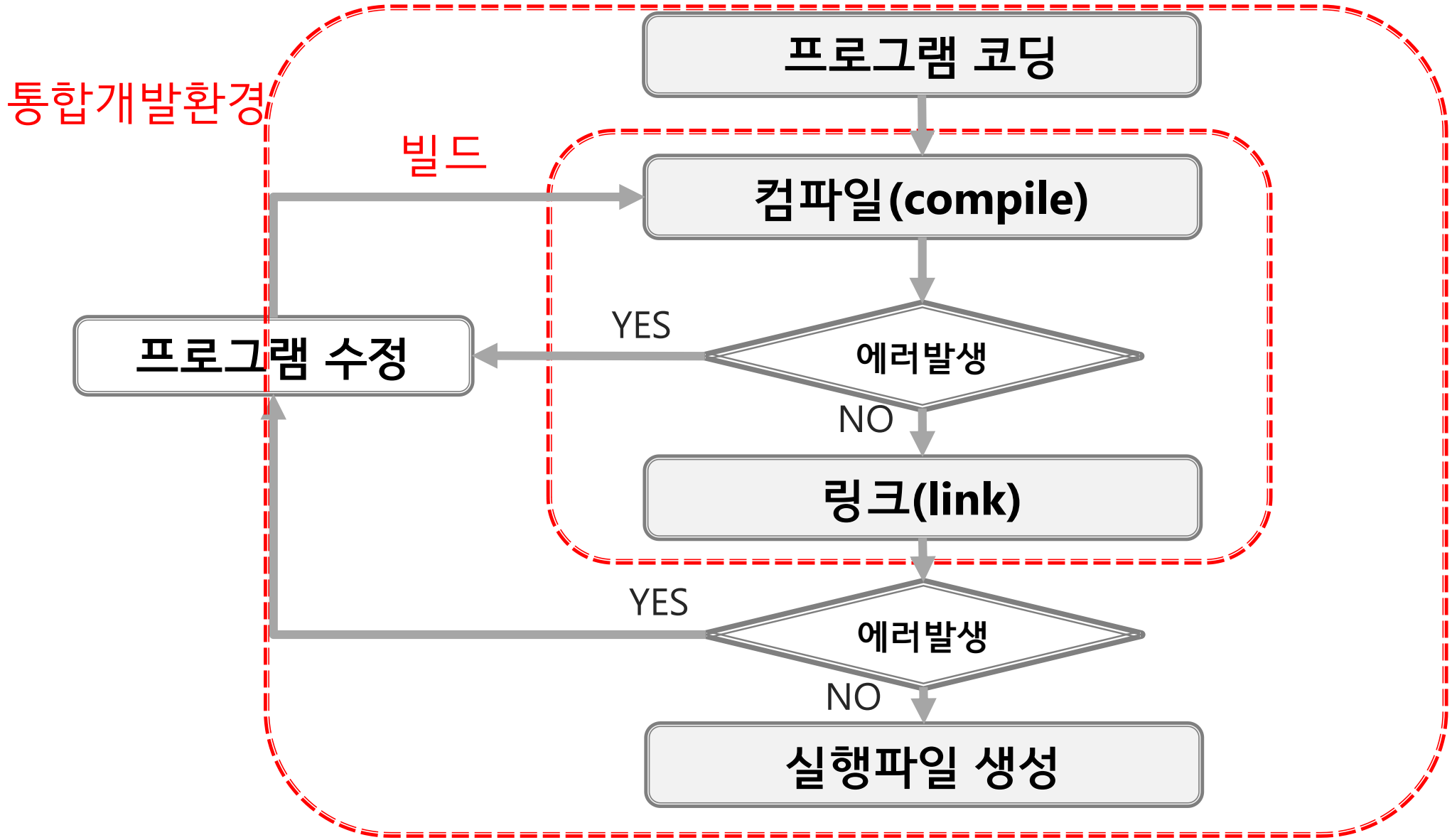
기계어로 번역



프로그래밍 언어


editor
compiler
linker
소스파일
(파일명.cpp)
compile
목적파일
(파일명.obj)
link
실행파일
(파일명.exe)
시스템내장
라이브러리
사용자지정
라이브러리

프로그래밍 언어


통합개발환경
빌드
프로그램 코딩
컴파일(compile)
YES
에러발생
프로그램 수정
NO
링크(link)
YES
에러발생
NO
실행파일 생성

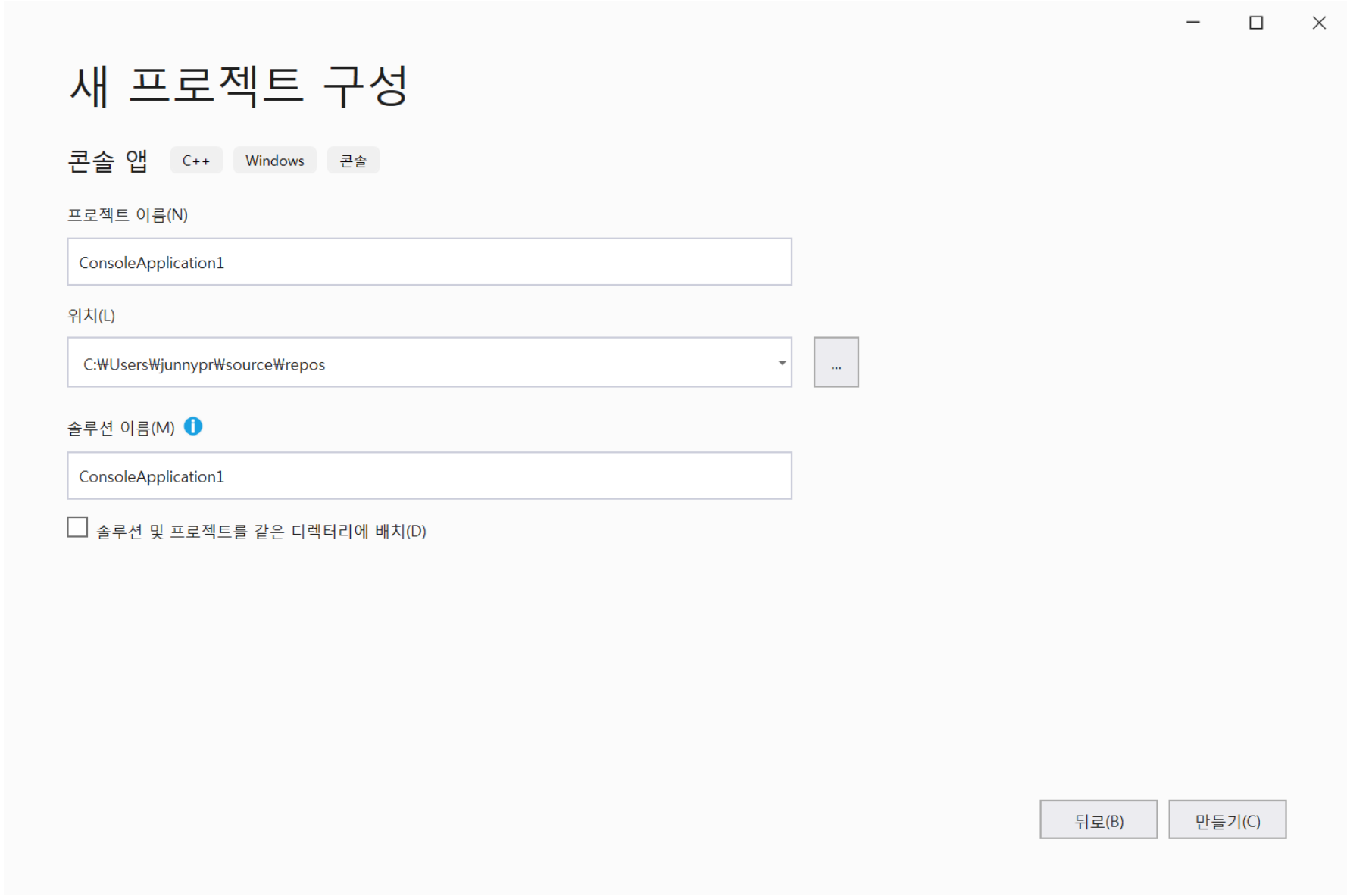
2강. 객체지향 프로그래밍 개요

Visual Studio 를 이용하여 프로그래밍하기

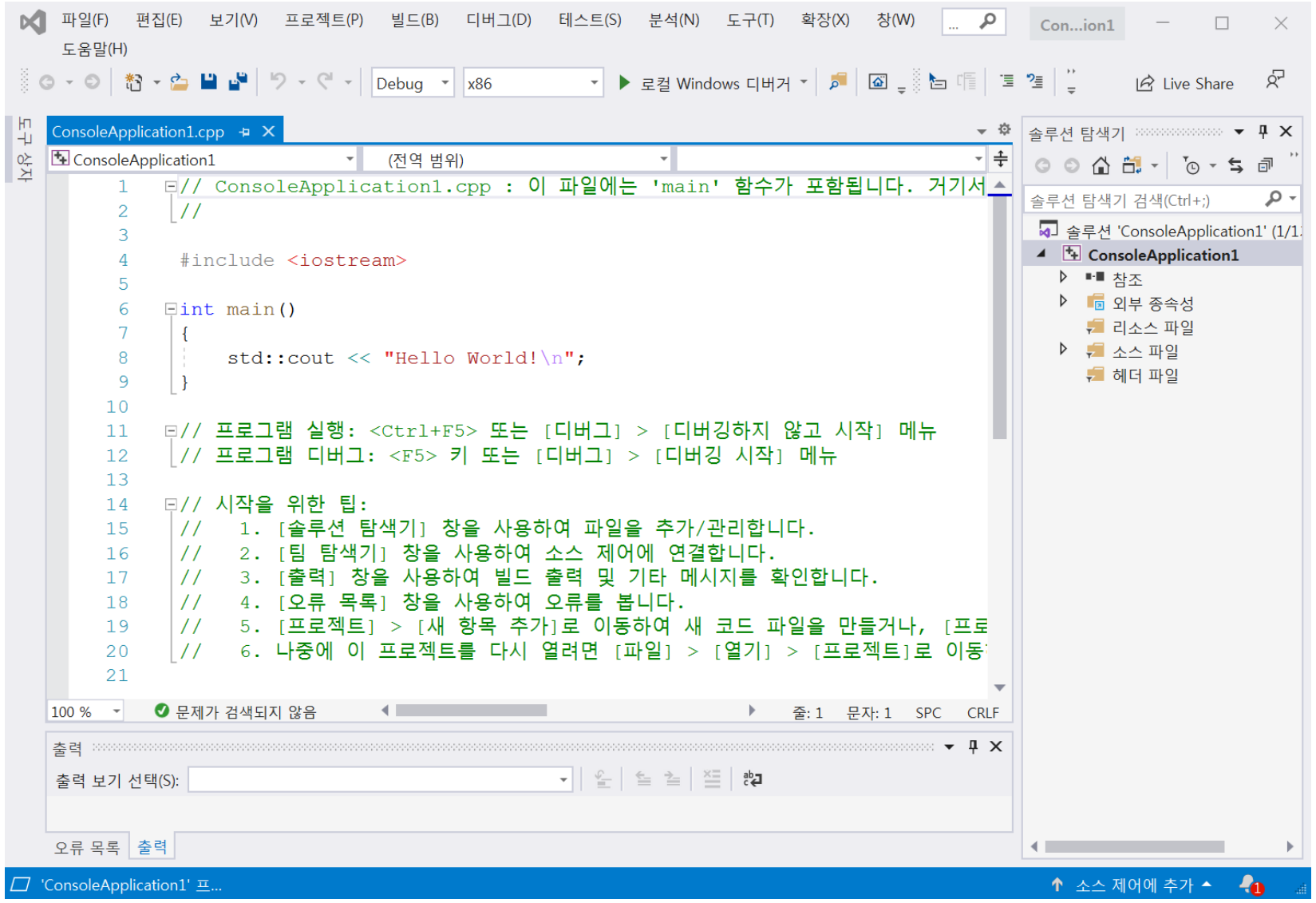
프로젝트 만들기(1)


새 프로젝트 만들기
템플릿 검색(Alt+S)(S)
최근 프로젝트 템플릿(R)
모든 언어(L)
모든 플랫폼(P)
모든 프로젝트 형식(T)
Windows 데스크톱 마법사 C++
빈 프로젝트 C++
콘솔 앱 C++
빈 프로젝트
Windows용 C++를 사용하여 처음부터 시작합니다. 시작 파일을 제공하지 않습니다.
C++ Windows 콘솔
콘솔 앱
Windows 터미널에서 코드를 실행합니다. 기본적으로 "Hello World"를 출력합니다.
C++ Windows 콘솔
Windows 데스크톱 마법사
마법사를 사용하여 고유한 Windows 앱을 만드세요.
C++ Windows 데스크톱 콘솔 라이브러리
Windows 데스크톱 애플리케이션
Windows에서 실행되는 그래픽 사용자 인터페이스를 사용하는 애플리케이션용 프로젝트입니다.
C++ Windows 데스크톱
공유 항목 프로젝트
공유 항목 프로젝트는 여러 프로젝트 간에 파일을 공유하는 데 사용됩니다.
C++ Windows Android iOS Linux 데스크톱 콘솔
다음(N)

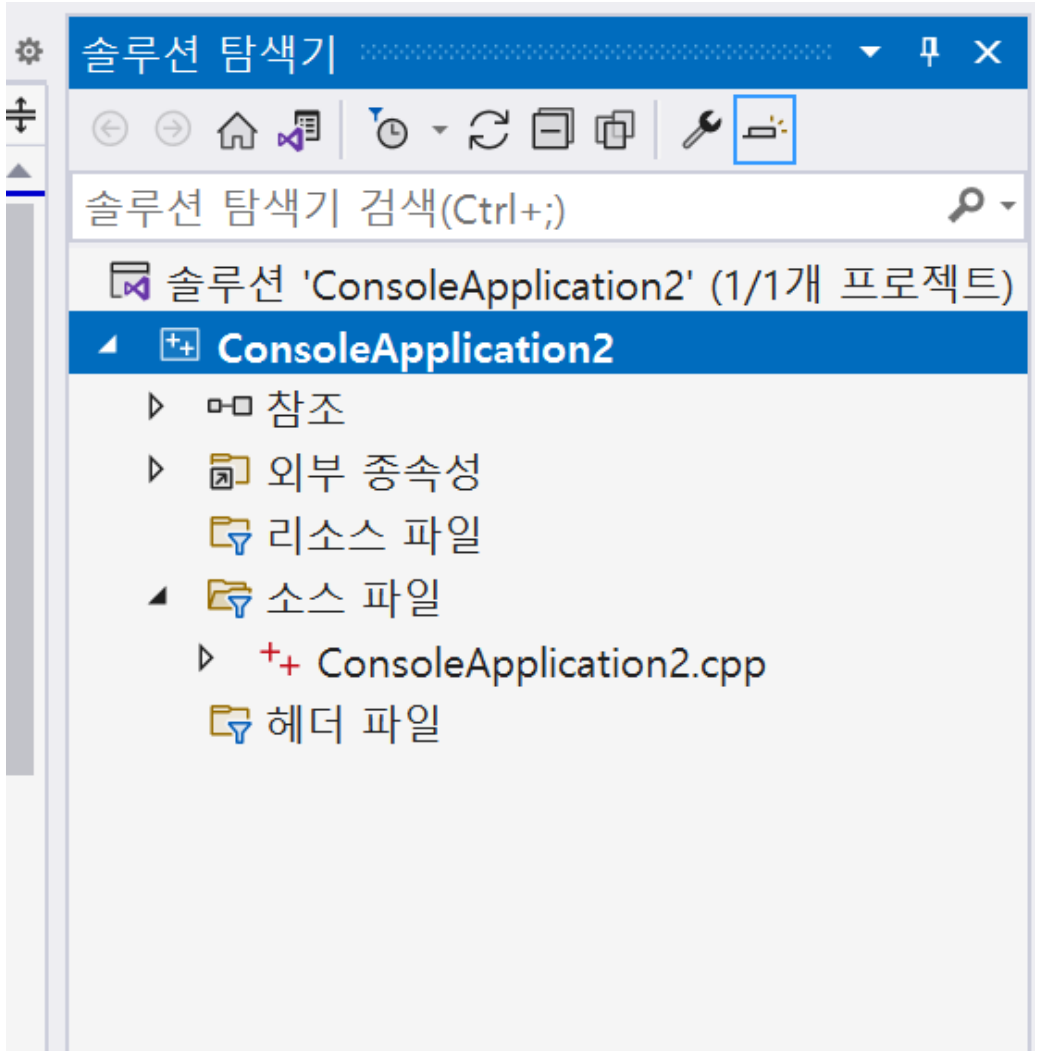
프로젝트 만들기(2)


새 프로젝트 구성
콘솔 앱
C++
Windows
콘솔
프로젝트 이름(N)
ConsoleApplication1
위치(L)
C:₩Users₩junnypr₩source₩repos
...
솔루션 이름(M)
ConsoleApplication1
솔루션 및 프로젝트를 같은 디렉터리에 배치(D)
뒤로(B)
만들기(C)

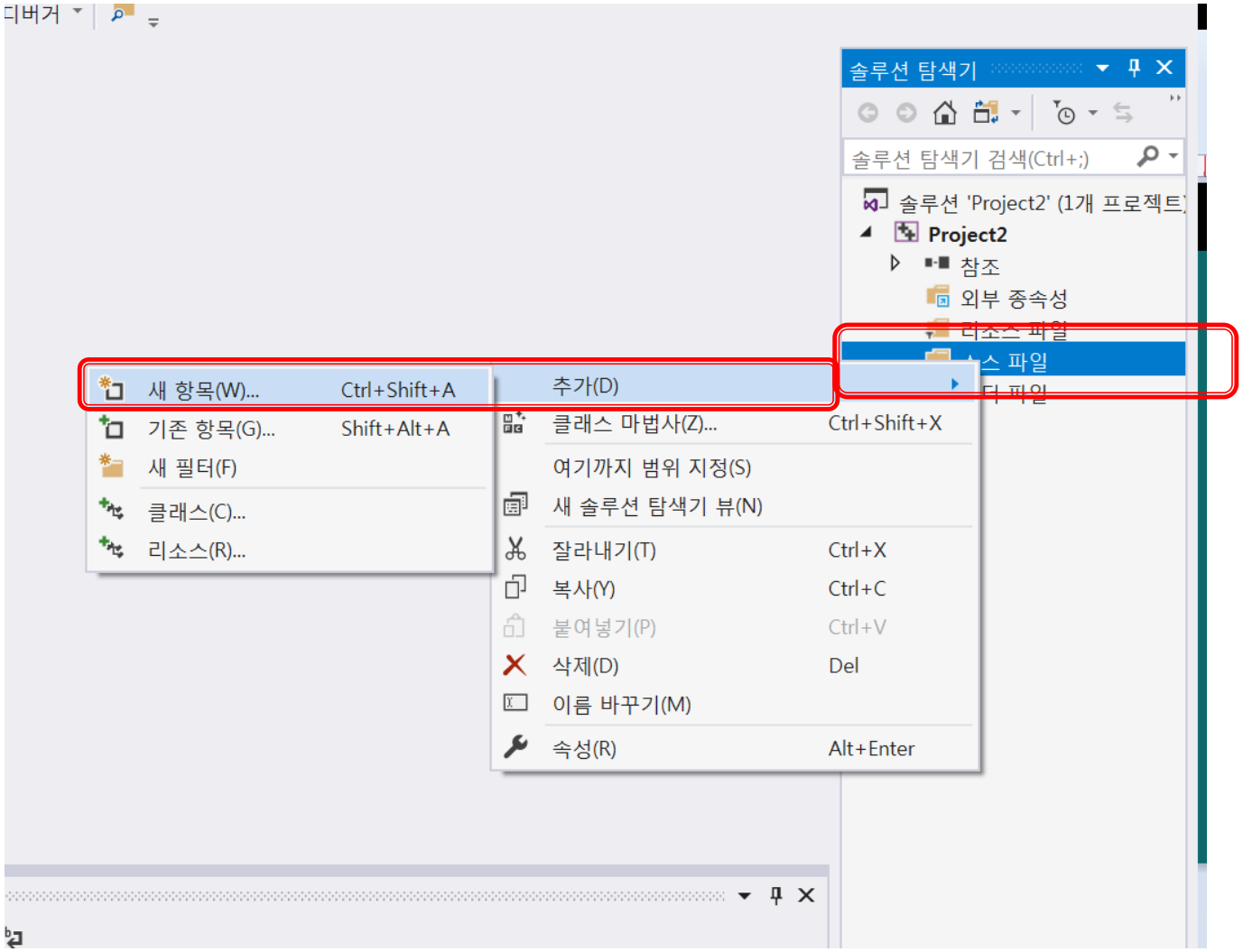
프로젝트 만들기(3)


파일(F) 편집(E) 보기(V) 프로젝트(P) 빌드(B) 디버그(D) 테스트(S) 분석(N) 도구(T) 확장(X) 창(W)
도움말(H)
Debug
x86
로컬 Windows 디버거
Live Share
ConsoleApplication1.cpp
ConsoleApplication1
(전역 범위)
// ConsoleApplication1.cpp : 이 파일에는 'main' 함수가 포함됩니다. 거기서
//
#include <iostream>
int main()
{
std::cout << "Hello World!\n";
}
// 프로그램 실행: <Ctrl+F5> 또는 [디버그] > [디버깅하지 않고 시작] 메뉴
// 프로그램 디버그: <F5> 키 또는 [디버그] > [디버깅 시작] 메뉴
// 시작을 위한 팁:
// 1. [솔루션 탐색기] 창을 사용하여 파일을 추가/관리합니다.
// 2. [팀 탐색기] 창을 사용하여 소스 제어에 연결합니다.
// 3. [출력] 창을 사용하여 빌드 출력 및 기타 메시지를 확인합니다.
// 4. [오류 목록] 창을 사용하여 오류를 봅니다.
// 5. [프로젝트] > [새 항목 추가]로 이동하여 새 코드 파일을 만들거나, [프로
// 6. 나중에 이 프로젝트를 다시 열려면 [파일] > [열기] > [프로젝트]로 이동
100 %
문제가 검색되지 않음
줄: 1 문자: 1 SPC CRLF
출력
출력 보기 선택(S):
오류 목록
출력
솔루션 탐색기
솔루션 탐색기 검색(Ctrl+;)
솔루션 'ConsoleApplication1'
ConsoleApplication1
참조
외부 종속성
리소스 파일
소스 파일
헤더 파일
'ConsoleApplication1' 프...
소스 제어에 추가

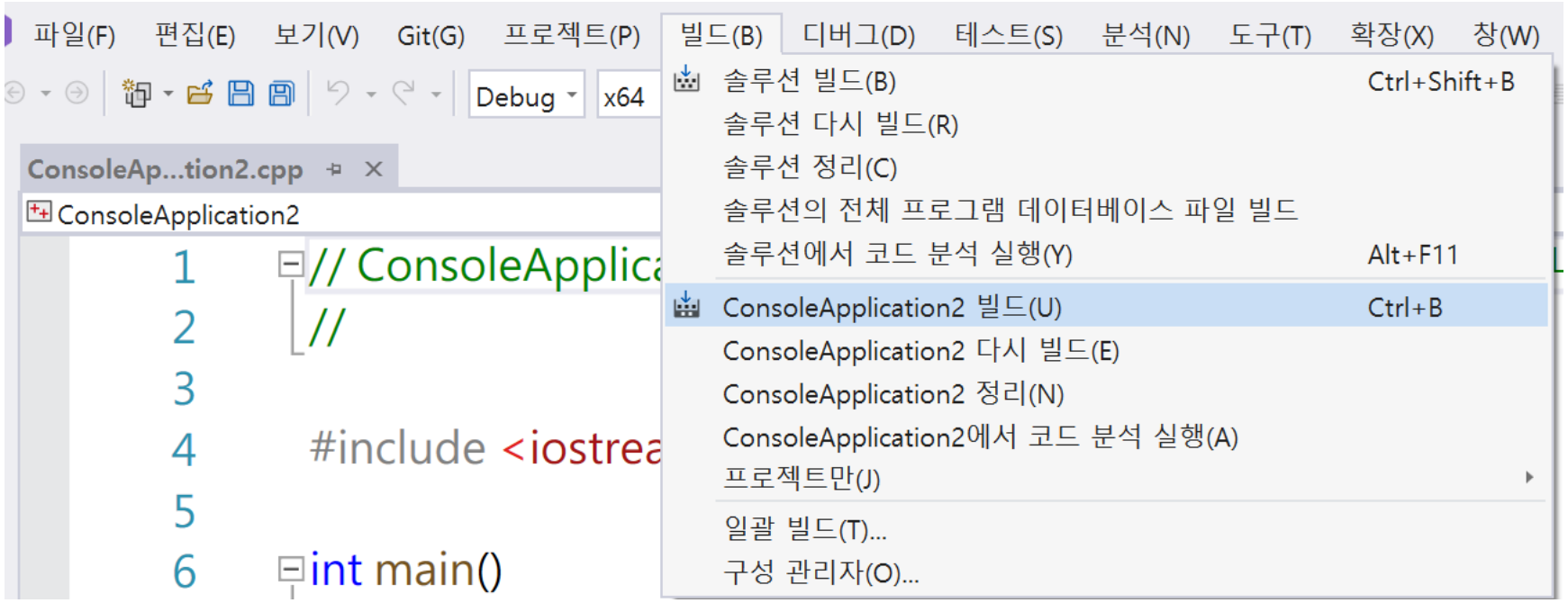
프로젝트의 구성


솔루션 탐색기
솔루션 탐색기 검색(Ctrl+;)
솔루션 'ConsoleApplication2' (1/1개 프로젝트)
ConsoleApplication2
참조
외부 종속성
리소스 파일
소스 파일
ConsoleApplication2.cpp
헤더 파일

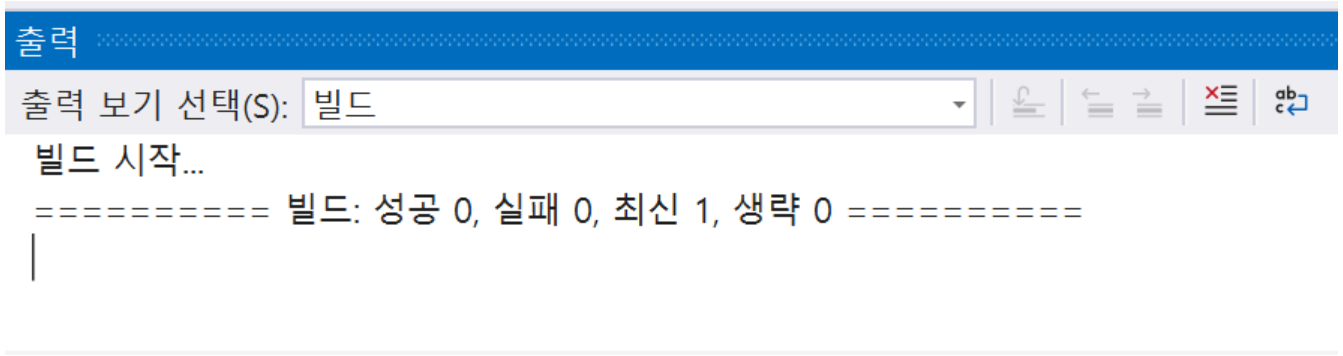
프로젝트의 구성


디버거
솔루션 탐색기
솔루션 탐색기 검색(Ctrl+;)
솔루션 'Project2' (1개 프로젝트
Project2
참조
외부 종속성
새 항목(W)... Ctrl+Shift+A
기존 항목(G)... Shift+Alt+A
새 필터(F)
클래스(C)...
리소스(R)...
추가(D)
클래스 마법사(Z)... Ctrl+Shift+X
여기까지 범위 지정(S)
새 솔루션 탐색기 뷰(N)
잘라내기(T) Ctrl+X
복사(Y) Ctrl+C
붙여넣기(P) Ctrl+V
삭제(D) Del
이름 바꾸기(M)
속성(R) Alt+Enter

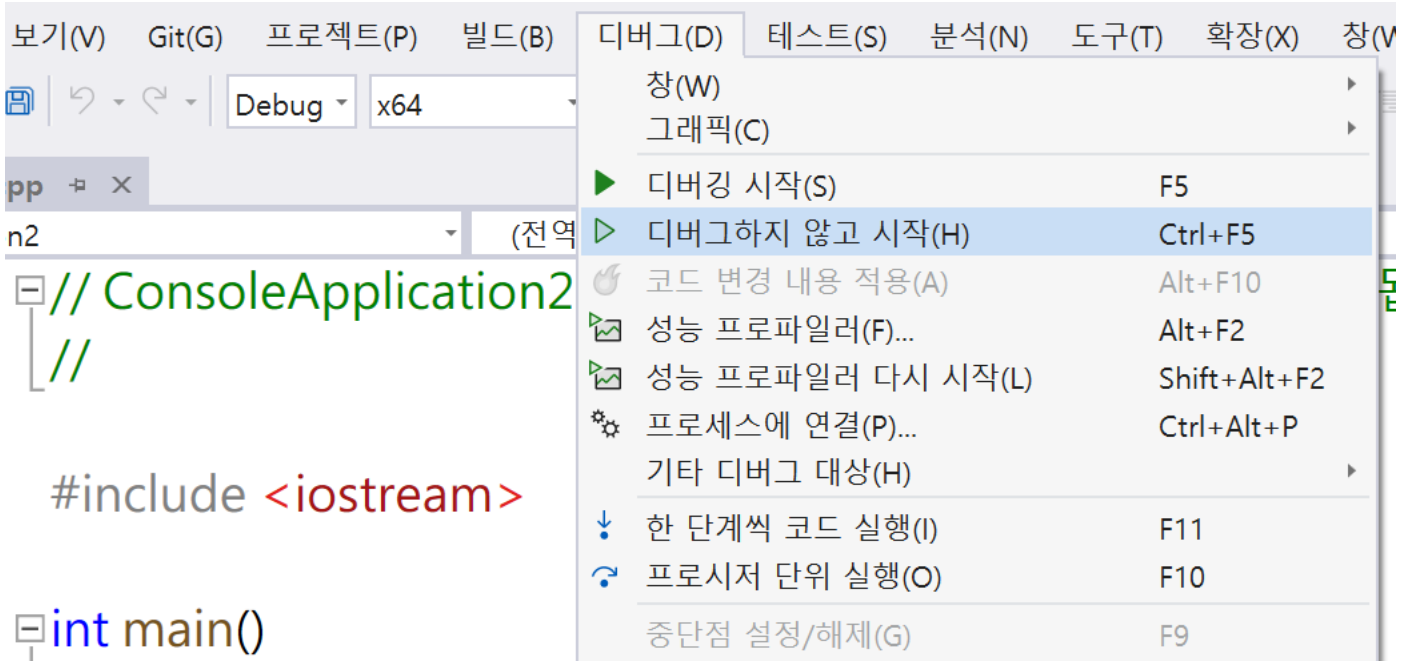
컴파일(빌드)



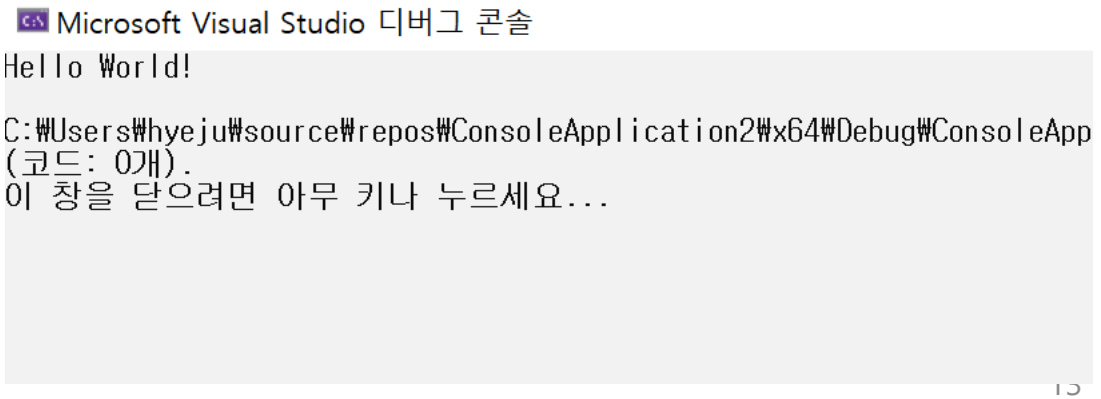
Ctrl + Shift + B



실행 - 디버그모드에서의 실행



Ctrl + F5



Hello World

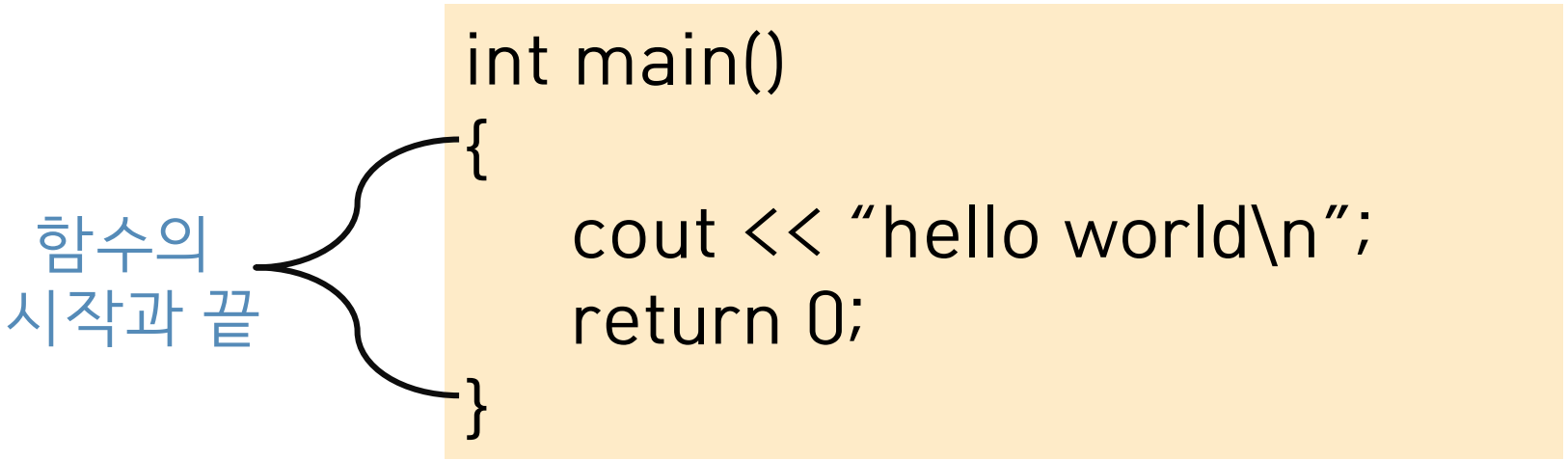
```
// myfirst.cpp – 메시지를 출력하는 프로그램

#include <iostream>

int main()
{
    std::cout << "Hello World!";
    std::cout << endl;
    std::cout << “C++ 세상에 온 것을 환영합니다.";
    return 0;
}
```

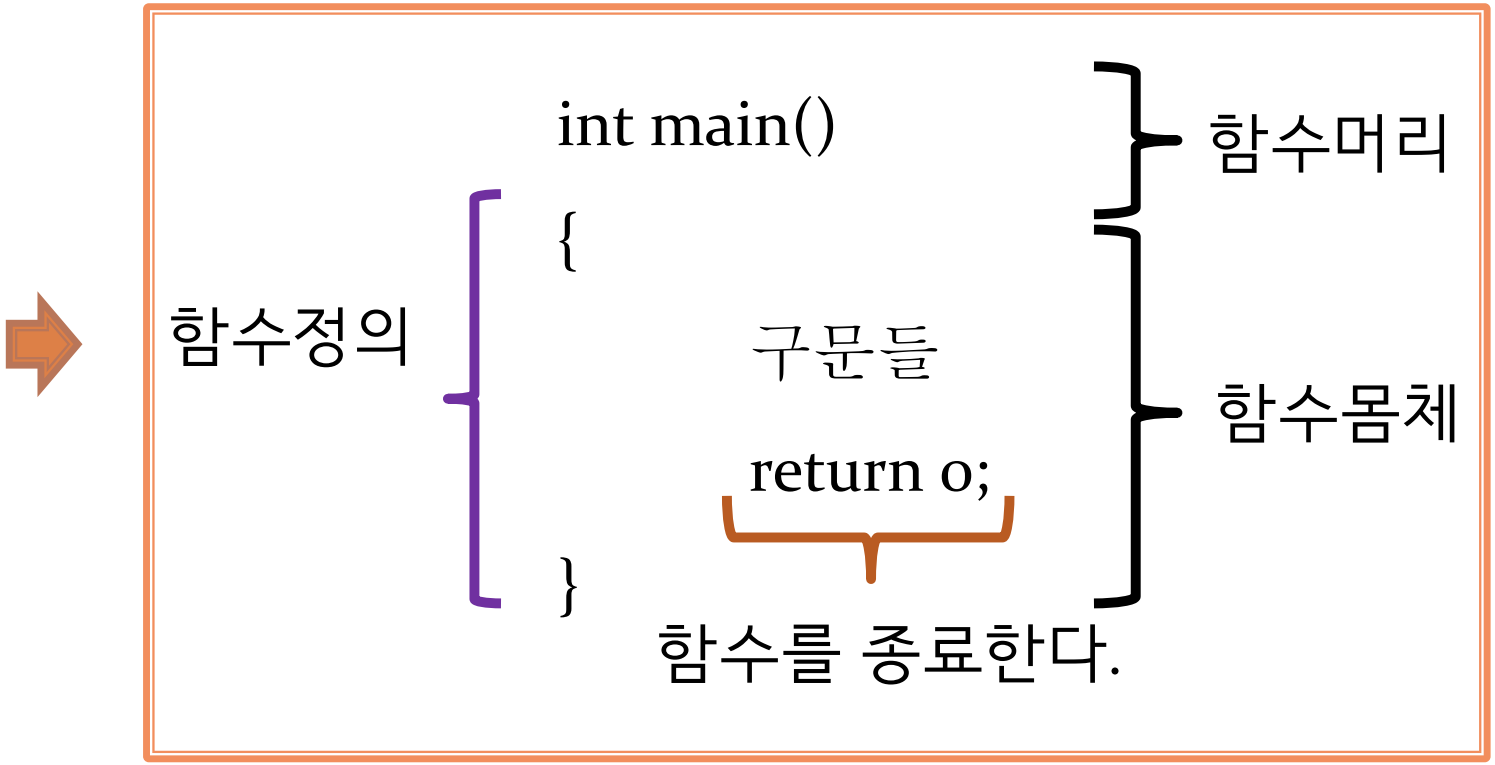
프로그램 구조 : main() 함수

- main()은 프로그램이 처음 시작될 때 호출되는 함수이다.
- 함수는 '{'로 시작하고 '}'로 끝난다.
- 문장은 위에서부터 아래로 차례로 실행된다.



프로그램 구조 : main() 함수

```
int main()
{
        구문들
        return 0;
}
```



세미콜론(;)은 구문(instruction)의 종결자이며 구문의 일부이다.
구문의 끝에 있는 세미콜론을 절대 빠트려서는 안된다.

프로그램 구조 : 전처리기

```
#include <iostream>
```

- 전처리기는 컴파일하기 전에 소스 파일에 대해 미리 어떤 처리를 수행하는 프로그램이다.
- 전처리 지시자는 #으로 시작한다.
- 전처리기는 소스파일을 컴파일할 때 자동으로 실행된다.
- #include <iostream> : 이 지시자는 전처리기에게 iostream 파일의 내용을 프로그램에 추가하라고 지시한다.
- iostream 헤더파일에는 C++의 입출력 기능이 정의되어 있다. 프로그램에서 cout 기능을 사용하려면 이러한 정의가 필요하다.

프로그램 구조 : 출력문

```
std::cout << "hello world";
```

- std::cout << " 출력할 내용 ";
- << : 문자열을 cout에 전달한다. 방향은 정보의 흐름을 나타낸다.
- cout : 다양한 정보들을 출력하는 미리 정의된 객체이다.

```
std::cout << endl;
```

- 화면 커서가 다음 행의 시작 위치로 감으로써 행이 바뀐다.
- iostream 헤더 파일에 정의되어 있고 std 이름 공간에 속한다.

프로그램 구조 : 출력문

\n : 개행문자(newline character)

- cout << “목성은 소형 행성이다. \n”;
- 출력에 새로운 행을 나타낸다.
- \n은 개행문자라고 부르는 하나의 문자를 나타내는 것으로 간주한다.

프로그램 구조 : 이름 공간

```
01  //Hello
02  #include <iostream>
03  using namespace std;
04
05  int main()
06  {
07      cout << “Hello World\n”;
08      return 0;
09  }
```

프로그램 구조 : namespace

- 하나의 프로젝트 내에서 이름 충돌 현상이 발생하지 않도록 C++에서는 namespace를 제공한다.
- 가령, 다른 사람이 각각 만든 sum()이 있을 경우, sum() 함수를 구별하기 위해 **"*namespace*::sum()"**의 형식을 사용한다.
- C++의 모든 표준 라이브러리는 **std**라는 namespace에 만들어져 있다.

```
std::cout<<"hello world";
```

- using 지시자를 사용하여 std:: 같은 접두어를 생략할 수 있다.

```
using namespace std;
cout<<"hello world";
```

프로그램 구조 : 주석문

- 더블슬래시(//) 뒤에는 항상 주석문이 온다.
- 주석문은 프로그래머가 프로그램안에 기록해 두는 일종의 메모이다.
- 프로그램의 구역을 구분하거나 코드의 어떤 부분이 무슨 역할을 하는 것인지 표시하는데 사용한다.

```
// myfirst.cpp – 메시지를 출력하는 프로그램
```

주석문을 사용하여 프로그램을 최대한 문서화해야 한다.
프로그램이 복잡해질수록 주석문이 중요하다.

프로그램 구조 : 제어 문자열

```
#include <iostream>

int main()
{
    std::cout <<"책제목은 \"게임으로 배우는 C++\"입니다.\n";
    return 0;
}
```

\n 줄바꿈
\t 탭문자
\\ 역슬래시
\' 작은 따옴표
\" 큰 따옴표

프로그램 작성 규칙

프로그램은 알아보기 쉽게 작성해야 한다.

```
int main()
{int English_score;
   English_score = 100; std::cout
<< English_scroe << endl;return 0;}
```

```
int main()
{
   int English_score;
   English_score = 100;
   std::cout << English_scroe << endl;
   return 0;
}
```

프로그램 작성 규칙

- 한 행에 하나의 구문을 사용한다.
- 함수를 여는 중괄호 {와 닫는 중괄호 }에 각각 한 행을 할애한다.
- 함수 안에 들어갈 구문들은 중괄호에서 약간 오른쪽으로 들어간 위치에서 시작한다.

나의 프로그램 작성 방법

```
// myfirst.cpp – 메시지를 출력하는 프로그램

#include <iostream>
using namespace std;

int main()
{

    return 0;
}
```

프로그램 오류 읽기 및 수정

프로그램 오류의 종류

① 컴파일 오류

- 프로그램의 문법적 오류가 발생하는 경우

② 런타임 오류(실행 오류)

- 문법적 오류가 없어 컴파일은 되나, 프로그램 실행 시, 오류가 발생
- 메모리 참조 오류, 배열 오류 등..

③ 로직 오류

컴파일 오류, 런타임 오류는 없으나 원하는 결과가 나오지 않는 경우

컴파일 오류

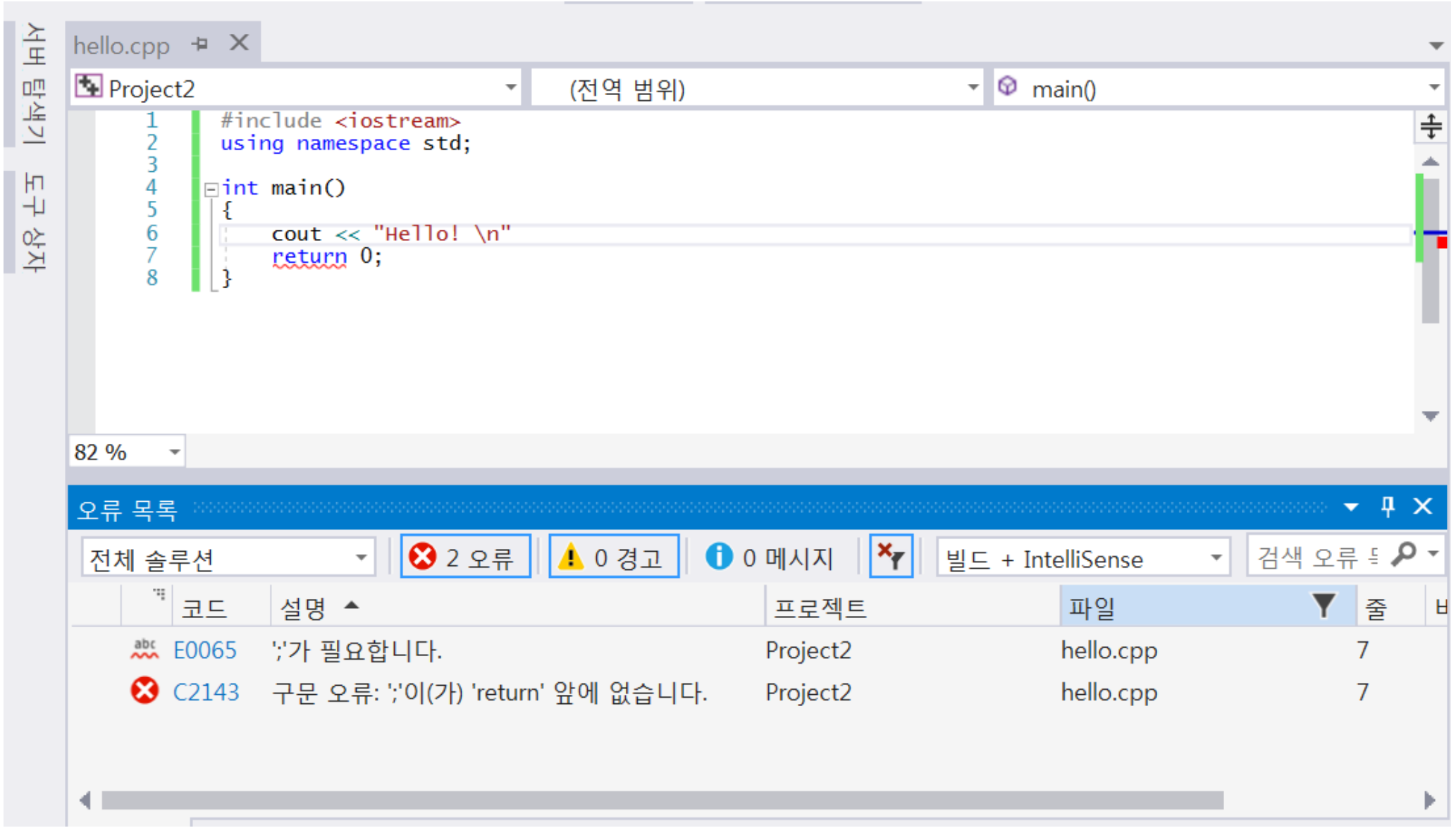
```
#include <iostream>
using namespace std;

int main()
{
  cout << “hello world\n”;
  return 0;
}
```



;를 삭제하고 컴파일해보기

컴파일 오류


서버 탐색기
도구 상자
hello.cpp
Project2
(전역 범위)
main()
1 #include <iostream>
2 using namespace std;
3
4 int main()
5 {
6 cout << "Hello! \n"
7 return 0;
8 }
82 %
오류 목록
전체 솔루션
2 오류
0 경고
0 메시지
빌드 + IntelliSense
검색 오류
코드
설명
프로젝트
파일
줄
E0065
';'가 필요합니다.
Project2
hello.cpp
7
C2143
구문 오류: ';'이(가) 'return' 앞에 없습니다.
Project2
hello.cpp
7

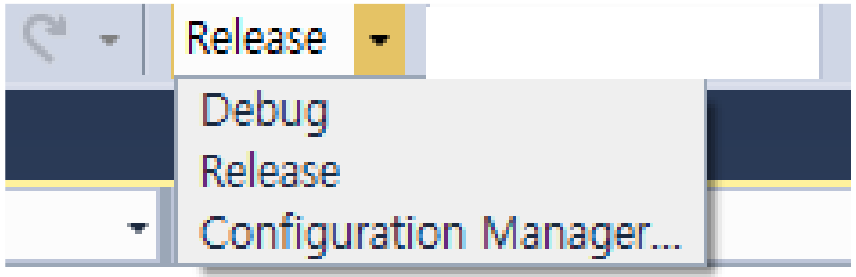
디버깅(Debugging)

- 프로그램 개발 단계 중에 발생하는 오류나 비정상적인 연산을 찾아내어 수정하는 과정
- 디버깅은 프로그래머가 되기 위한 초석
- 프로그램을 조사하면서 프로그램이 실행되는 과정을 들여다 보는 것이 중요
- 프로그램을 살펴보는 방법
    - 디버깅 도구 사용
    - 코드 줄 출력 : 정보를 출력하는 줄을 추가
    - 로깅(logging) : 프로그램 실행을 로그의 형태로 확인하는 것

디버그(Debug)와 릴리즈(Release)

- Debug 모드
  - ✓ 실행 파일에 디버깅 정보를 삽입하여 언제든지 디버깅 가능
  - ✓ 디버깅 정보가 포함되어 실행파일 크기가 크다.
  - ✓ 디버그에 필요한 정보를 계속 확인하므로 실행 시 상대적으로 느림

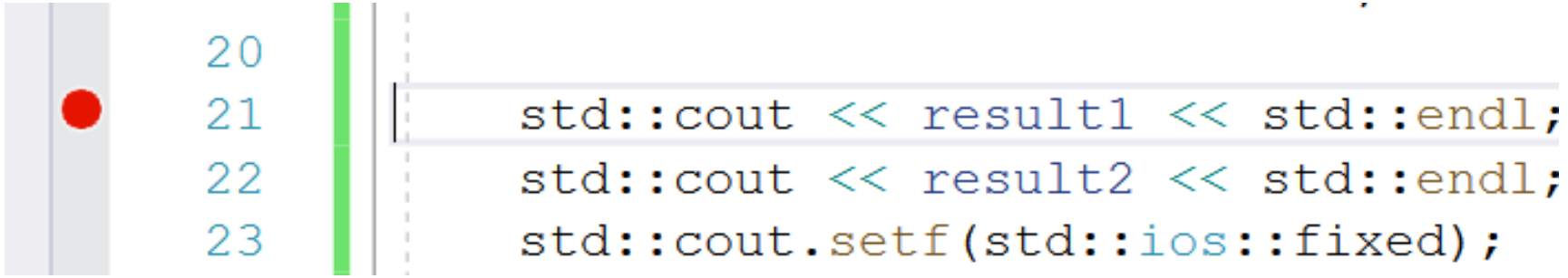
- Release 모드
  - ✓ 코드 최적화
  - ✓ 실행파일 크기가 Debug 모드보다 작으며 속도도 더 빠름



디버깅 방법 - ①중단점(Break Point) 설정

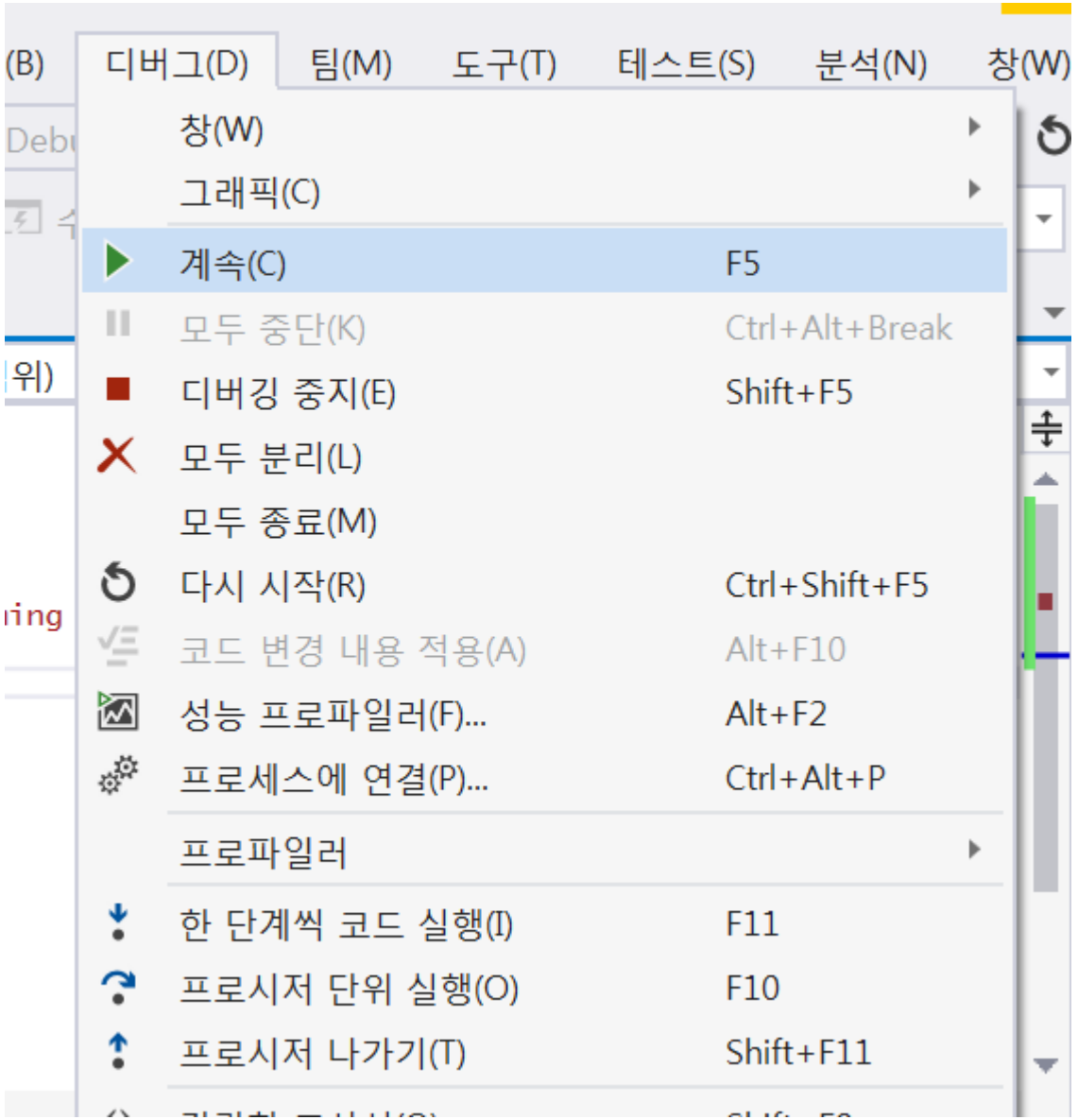
- 소스 코드의 특정 지점에서 프로그램 실행을 멈추는데 사용

- 중단점 설정 방법
  - F9 또는 해당 코드의 왼쪽바에서 마우스를 클릭해서 설정

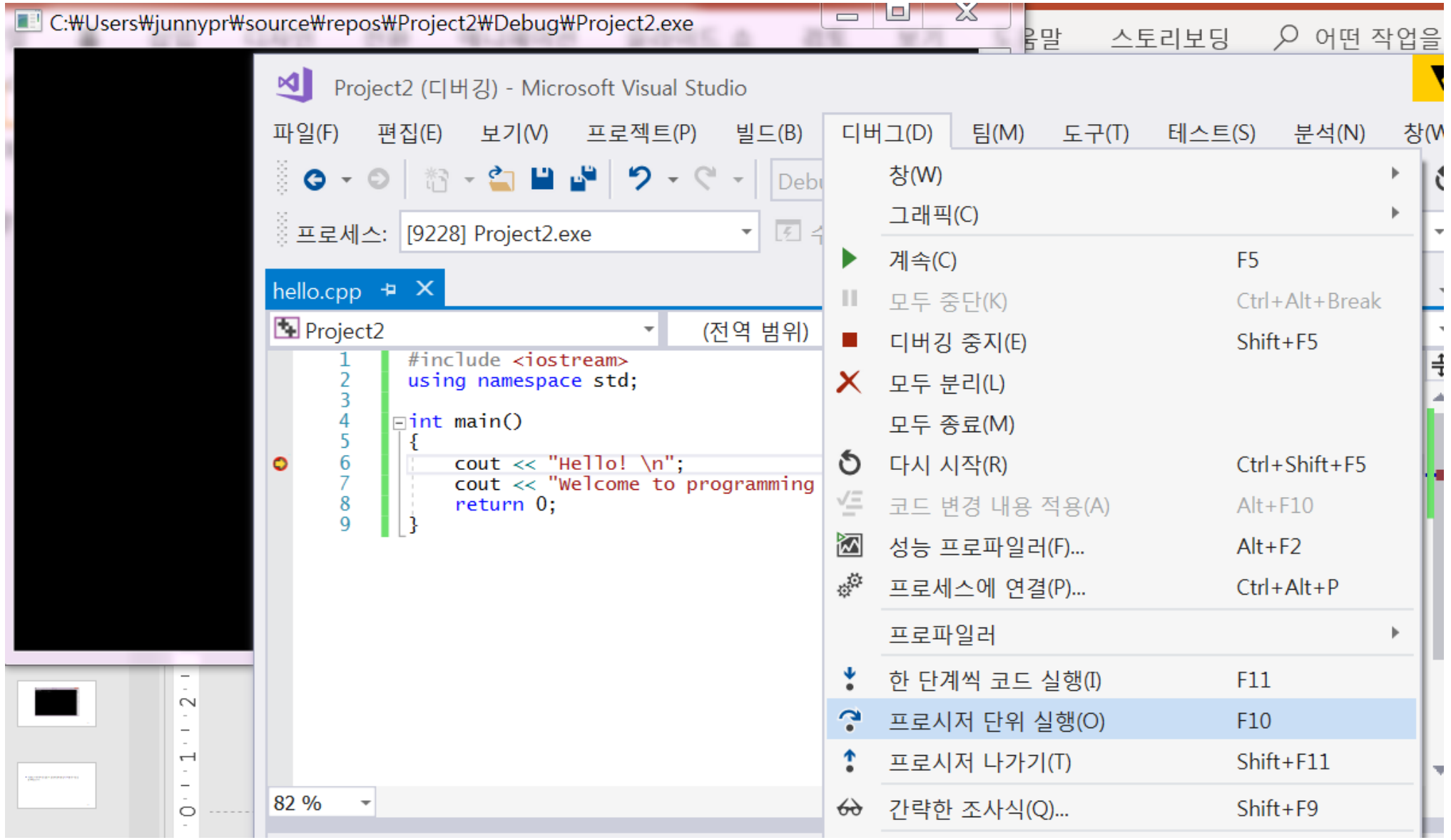


디버깅 방법 – ②코드 이동

- 디버깅 시작 : F5
- 코드 실행
  - F10 : 현재 소스코드에서 한 줄씩
  - 실행
  - F11 : 프로시저단위로 실행
  - F5 : 다음 중단점까지 이동(디버깅 중)
- 디버깅 끝 : Shift+F5

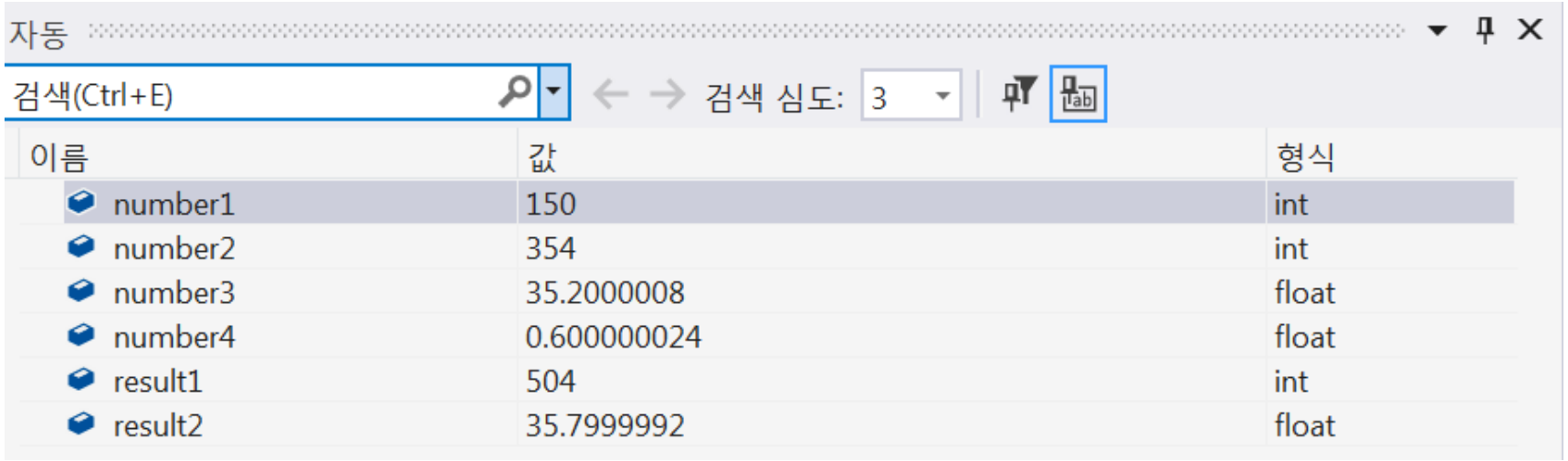


디버깅 방법 - ②코드 이동


C:₩Users₩junnypr₩source₩repos₩Project2₩Debug₩Project2.exe
Project2 (디버깅) - Microsoft Visual Studio
파일(F) 편집(E) 보기(V) 프로젝트(P) 빌드(B) 디버그(D) 팀(M) 도구(T) 테스트(S) 분석(N)
프로세스: [9228] Project2.exe
hello.cpp
Project2
(전역 범위)
#include <iostream>
using namespace std;
int main()
{
cout << "Hello! \n";
cout << "Welcome to programming
return 0;
}
82 %
창(W)
그래픽(C)
계속(C) F5
모두 중단(K) Ctrl+Alt+Break
디버깅 중지(E) Shift+F5
모두 분리(L)
모두 종료(M)
다시 시작(R) Ctrl+Shift+F5
코드 변경 내용 적용(A) Alt+F10
성능 프로파일러(F)... Alt+F2
프로세스에 연결(P)... Ctrl+Alt+P
프로파일러
한 단계씩 코드 실행(I) F11
프로시저 단위 실행(O) F10
프로시저 나가기(T) Shift+F11
간략한 조사식(Q)... Shift+F9

디버깅 방법- ③Watch

- 애플리케이션의 변수값을 확인하여 정상적으로 실행되는지 여부를 확인
- Debug 모드일 경우에만 볼 수 있음



[예제1] 이름과 주소를 출력하는 프로그램을 작성하라.